

1. Concurrency Control Motivation

ACID Isolation

- Coordinates concurrent execution to avoid 4 major anomalies:
- **Dirty Read (WR):** Reading uncommitted data that later aborts.
 - **Lost Update (WW):** Two writes overwrite without intermediate visibility.
 - **Unrepeatable Read (RW):** Reading different values in same txn.
 - **Phantom Read:** Range query tuple count changes via inserts.

2. Lock Types & Compatibility

Lock Manager

- **Binary Lock (0/1):** 0 = Unlocked, 1 = Locked. Only 1 txn allowed (Bottleneck).
- **Shared Lock S(X):** Read permission. Multiple readers allowed.
- **Exclusive Lock X(X):** Read and Write permission. Strictly 1 txn.

Held \ Requested	Shared (S)	Exclusive (X)
Shared (S)	TRUE (Compatible)	FALSE (Conflict)
Exclusive (X)	FALSE (Conflict)	FALSE (Conflict)

3. Two-Phase Locking (2PL) Protocol & Lock Point

Serializability Theorem

Phase 1: Growing Phase (Locks acquired only; no release)

Lock Point: Instant when the LAST lock is acquired

Phase 2: Shrinking Phase (Locks released only; no acquire)

Golden Rule: Once a transaction releases any lock, it can NEVER acquire any new lock!

Lock Conversion Rules:

- **Upgrading (S → X):** Allowed ONLY in Growing Phase if T is sole reader.
- **Downgrading (X → S):** Allowed in Shrinking Phase to increase read concurrency.

Serial Order: Equivalent serial order equals the order of transactions reaching their Lock Points.

4. Master 2PL Protocol Variants Comparison Matrix

AKTU 10M Question

Variant	Lock Holding & Release Rule	Deadlock-Free?	Cascadeless?	Industry Role
Basic 2PL	Growing phase then Shrinking phase. Locks released anytime in phase 2.	No (Deadlock possible)	No (Cascading abort possible)	Theoretical baseline
Conservative 2PL	Pre-declares and locks ALL required items before execution starts.	YES (Guaranteed!)	No	Static/Embedded DBs
Strict 2PL	All Exclusive (X) locks held until Commit / Abort . Shared locks released in phase 2.	No (Deadlock possible)	YES (Strict & Cascadeless)	Industry Standard (PostgreSQL, MySQL)
Rigorous 2PL	All Shared (S) and Exclusive (X) locks held until Commit / Abort .	No (Deadlock possible)	YES (Commit Order CSR)	Financial engines

5. Basic Timestamp Ordering (TO)

Deadlock-Free

TS(T) assigned at start. Item timestamps: read_TS(X) & write_TS(X).

read(X): If $TS(T) < write_TS(X) \rightarrow$ ABORT T.
Else: execute and $read_TS(X) = \max(read_TS(X), TS(T))$.
write(X): If $TS(T) < read_TS(X)$ or $TS(T) < write_TS(X) \rightarrow$ ABORT T.
Else: execute and $write_TS(X) = TS(T)$.

Advantage: 100% Deadlock-Free (No waiting cycles!).

7. Multi-Version (MVCC) & MVTO

Non-Blocking Readers

Writes append new version X_k (val), RTS, WTS angle\$.

- Mantra:** Readers never block writers; writers never block readers!
- MVTO read(X):** Reads X_i with largest $WTS(X_i) \leq TS(T)$. **Read is ALWAYS granted (Never aborts)!**
- MVTO write(X):** If $RTS(X_i) > TS(T)$ implies\$ Abort. Else creates new version X_{k+1} with $WTS = RTS = TS(T)$.

9. Validation-Based Protocol (OCC) & Phantom Phenomena

The 3 Phases of OCC:

- Read Phase:** Reads DB freely; writes buffered locally in memory.
- Validation Phase:** Assigned $TS(T)$; validates conflict conditions.
- Write Phase:** Local copies flushed to DB disk upon validation pass.

Condition 2: $WritePhase(T_i) < WritePhase(T_j)$ ext{ and } $RS(T_j) \cap WS(T_i) = \emptyset$

6. Thomas' Write Rule & VSR

Obsolete Writes

Modifies Basic TO for write operations:

- If $TS(T) < read_TS(X) \rightarrow$ ABORT and ROLLBACK T.
- If $TS(T) < write_TS(X) \rightarrow$ **IGNORE write, CONTINUE processing!**
- Else $\rightarrow$ Execute write, $write_TS(X) = TS(T)$.

AKTU Theorem: Enforces **View Serializability (VSR)** using Blind Writes, rejecting fewer writes than Basic TO!

8. MV2PL & Certify Locks

3 Lock Modes

Employs Read Lock (RL), Write Lock (WL), Certify Lock (CL):

Held \ Req	RL	WL	CL
RL	YES	YES	NO
WL	YES	NO	NO
CL	NO	NO	NO

WL upgrades to CL at commit phase. Eliminates cascading rollbacks!

The Phantom Phenomenon:

- Predicate query count altered by concurrent dynamic tuple insertions.
- Existing row locks cannot protect non-existent records.
- Solution: Index-Range Locking** on B+ Tree leaf keys covering search range.

Optimistic CC

10. Granularity Tree Hierarchy

db → f → p → r

Tree: Database (\$db\$) → Files (\$f_1, f_2\$) → Pages (\$p_{11}, p_{12}\$) → Records (\$r_{111}\$).

- **Fine (Record):** High concurrency, high locking overhead.
- **Coarse (File):** Low concurrency, minimal locking overhead.
- **Subtree Invariant:** Locking node \$N\$ locks all its descendants!

11. Intention Locks & 5x5 Matrix (Slide 69)

IS, IX, SIX

Held \ Req	IS	IX	S	SIX	X
IS	Yes	Yes	Yes	Yes	No
IX	Yes	Yes	No	No	No
S	Yes	No	Yes	No	No
SIX	Yes	No	No	No	No
X	No	No	No	No	No

12. The 6 Mandatory Rules of MGL Protocol (Slide 70)

AKTU Exam Theorem

- **Rule 1:** Adhere strictly to 5×5 lock compatibility matrix.
- **Rule 2:** Root of tree (\$db\$) must be locked first, in any mode.
- **Rule 3 (S / IS):** Lock node \$N\$ in \$S\$ or \$IS\$ only if parent is locked in \$IS\$ or \$SIX\$.
- **Rule 4 (X / IX / SIX):** Lock node \$N\$ in \$X, IX, SIX\$ only if parent locked in \$IX\$ or \$SIX\$.
- **Rule 5 (2PL):** Lock only if no node has been unlocked (Growing phase).
- **Rule 6 (Bottom-Up):** Unlock \$N\$ only if NONE of its children are currently locked!

Direction: Locking is strictly TOP-DOWN | Unlocking is strictly BOTTOM-UP

13. Concurrent Recovery & 2PC

Distributed Atomicity

- **Concurrent Recovery:** Checkpoint records active list \$L\$. \$T_i\$ with commit → REDO (forward). \$T_i\$ active at crash → UNDO (backward).
- **Two-Phase Commit (2PC):**
 - **Phase 1 (Prepare):** Coordinator sends PREPARE → Nodes vote COMMIT or ABORT.
 - **Phase 2 (Commit):** If all vote COMMIT → GLOBAL_COMMIT; else GLOBAL_ABORT.

14. Oracle Case Study (AKTU Syllabus)

Industry Engine

- **MVCC & SCN:** System Change Number increments at commit.
- **Undo Segments:** Reconstructs Consistent Read (CR) buffer for older queries without blocking writers.
- **ORA-01555 Snapshot Too Old:** Occurs when query's required undo blocks are overwritten. Solution: Increase UNDO_RETENTION.
- **Zero Lock Escalation:** Row locks in block ITL; never escalates.